

Projenin Gelişim Süreci (Kronolojik ve Edilgen)

1. Veri Hazırlama (Preprocessing):

- Ham günlük (log) verileri temizlendi.
- Metin tabanlı boyut bilgileri Regex kullanılarak sayısal değerlere dönüştürüldü.
- Modelin verimli çalışabilmesi için **MinMaxScaler** ile tüm veriler 0-1 aralığında ölçeklendirildi.
- Çalışan kimlikleri, modele girdi olabilmesi için **LabelEncoder** yöntemiyle sayısal hale getirildi.

2. Model Mimarisinin Oluşturulması:

- Zaman serilerindeki davranış kalıplarını öğrenmesi amacıyla bir **LSTM Autoencoder** mimarisi tasarlandı.
- Veriler, 10 günlük zaman pencereleri (**window_size=10**) halinde gruplandırıldı.

3. Federe Öğrenme Altyapısı (Flower):

- Veri gizliliğini korumak amacıyla **Flower** kütüphanesi sisteme entegre edildi.
- **ServerApp** (sunucu) ve **ClientApp** (istemci) yapıları tanımlandı.
- İstemcilerin yerel ağırlıklarını sunucuya aktarmasını sağlayan **NumPyClient** yapısı kurgulandı.

4. Donanım Hızlandırma (GPU Kullanımı):

- Eğitim süresini kısaltmak amacıyla CPU yerine GPU (CUDA) kullanımına geçildi.
- PyTorch'un CUDA destekli sürümü yüklenerek donanım optimizasyonu sağlandı.

5. Eğitim ve Sonuçların Doğrulanması:

- Prototip eğitim süreci başarıyla tamamlandı. Üç round sonunda kayıp (loss) değerinin **0.0055'ten 0.0034'e** gerilediği ve modelin öğrenme sürecini başarıyla sürdürdüğü raporlandı.

Round sayısı 3 → 50 olarak değiştirildi ve eğitim yapıldı.

Ayrıca, eğitilen modellerin sonuçları (yani ağırlıklar) bir klasöre kaydedildi.

Eğitim sonrası modeli doğrulamak için bir “script” yazıldı. Yazılan “script”te test amaçlı eldeki verilerdeki bir veri değiştirilerek anormal davranış simüle edildi ve normal ve anormal veri üzerinden kontroller yapıldı.

Bu kontroller sonucu modelin anormal davranışları bazen saptayabildiği, ama bunu yeterince tutarlı olarak yapmadığı görüldü.

Örnek olarak, modelin 15 round eğitilmesi sonucu elde edilen ağırlıklar kullanıldı ve 0-1 aralığındaki normal verilerden birine “8” değeri verildi. Sonuç olarak aşağıdaki elde edildi:

--- Inference Results ---

Normal Sequence MSE: 0.211455

Anomalous Sequence MSE: 0.584157

Detection Ratio: 2.76x

Suggested Threshold: 0.634364

✓ STATUS: NORMAL BEHAVIOR

Görüldüğü gibi verinin anormal olmasına rağmen model bunu tam olarak fark edemedi.

Daha gerçekçi bir test olması için gerçek veri kullanılmasına karar verildi. Bu yüzden en son kullanılan test scripti tekrar yazıldı. Ayrıca tutarlılık olması için eğitimde kullanılan “MinMaxScaler” objesi “pickle” kullanılarak bir dosya olarak kaydedildi ve test yapılırken kullanılması sağlandı.

Gerçek veri kullanımının zor olması nedeniyle simülasyon kullanımına karar verildi. Test yapılırken eğitim verisindeki normal bir örüntüye kıyasla simüle edilen örüntülerin 20 kata kadar daha fazla “Mean Square Error” (MSE) değerine sahip olduğu görüldü.

Daha kapsamlı test yapılabilmesi için farklı saldırıların simüle edilmesi ve mümkünse gerçek veri kullanılmasına karar verildi. Simüle edilen saldırılarda da yüksek bir başarı elde edildiği görüldü.

Round | Normal MSE | Espionage (Ratio) | Exfil (Ratio) | Sabotage (Ratio)

0	0.015340	0.91 x	1.11 x	9.32 x
1	0.001178	1.53 x	11.09 x	108.29x
2	0.000170	12.11 x	53.89 x	732.91x

Hazırlanan “feature extraction” script’inin eksik veri çıkardığı ve dataset’teki “ground truth” olan kötü amaçlı kişilerin bilgilerini çıkarmadığı gözlemlendi. Bunun üzerine var olan bir script kullanılmasına karar verildi:

https://github.com/lcd-dal/feature-extraction-for-CERT-insider-threat-test-datasets/blob/main/feature_extraction.py

Bu script’in eski bir pandas sürümü için yazılmasından ötürü script üzerinde bazı güncellemeler yapıldı.

508 tane özellik çıkartıldı. Modeli eğitmek için bu özellikler 0-1 aralığında normalize edildi (MinMaxScaler kullanılarak). Kullanıcı faaliyetlerinin zamansal bağlamını korumak amacıyla veriler kronolojik olarak sıralanmış ve "Sliding Window" (Kayan Pencere) tekniği uygulanmıştır. Her bir veri örneği, 10 ardışık işlem adımından oluşan zaman serisi blokları (sequences) haline getirilmiştir. Bu yapı, modelin sadece anlık olayları değil, olaylar arasındaki zamansal ilişkileri de öğrenmesine olanak sağlamıştır.

Federe Öğrenme için Flower kütüphanesi kullanıldı. Veri seti 10 farklı istemciye (client) paylaştırılmış, eğitim süreci merkezi bir sunucu yerine bu yerel istemciler üzerinde gerçekleştirilmiştir. Sunucu tarafında FedAvg (Federated Averaging) algoritması kullanılarak yerel model ağırlıkları birleştirilmiş ve kolektif bir "Küresel Model" (Global Model) elde edilmiştir.

Anomali tespiti için derin öğrenme tabanlı bir LSTM-Autoencoder mimarisi tasarlanmıştır. Model, eğitim aşamasında sadece normal kullanıcı davranışlarını yeniden yapılandırmak (reconstruction) üzere 20 iterasyon (round) boyunca eğitilmiştir. Eğitim sonunda modelin kayıp (loss) değerinin 0.014 seviyelerine kadar düştüğü ve kararlı bir yapıya kavuştuğu gözlemlenmiştir.

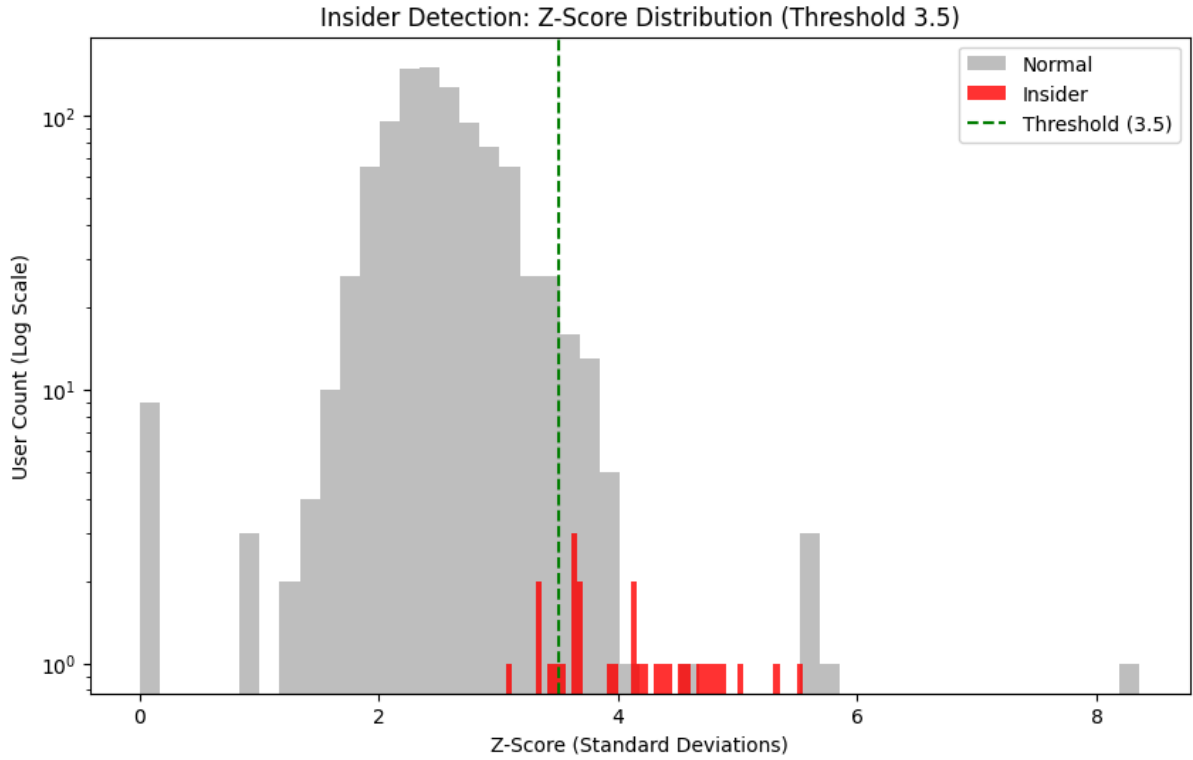
Sistem yöneticileri gibi doğal olarak yüksek hacimli işlem yapan "gürültülü" kullanıcıların yanlış alarm (false positive) üretmesini engellemek için Yerel Z-Skoru yöntemi geliştirilmiştir. Her kullanıcının yeniden yapılandırma hata payı, kendi geçmiş ortalaması ve standart sapmasıyla

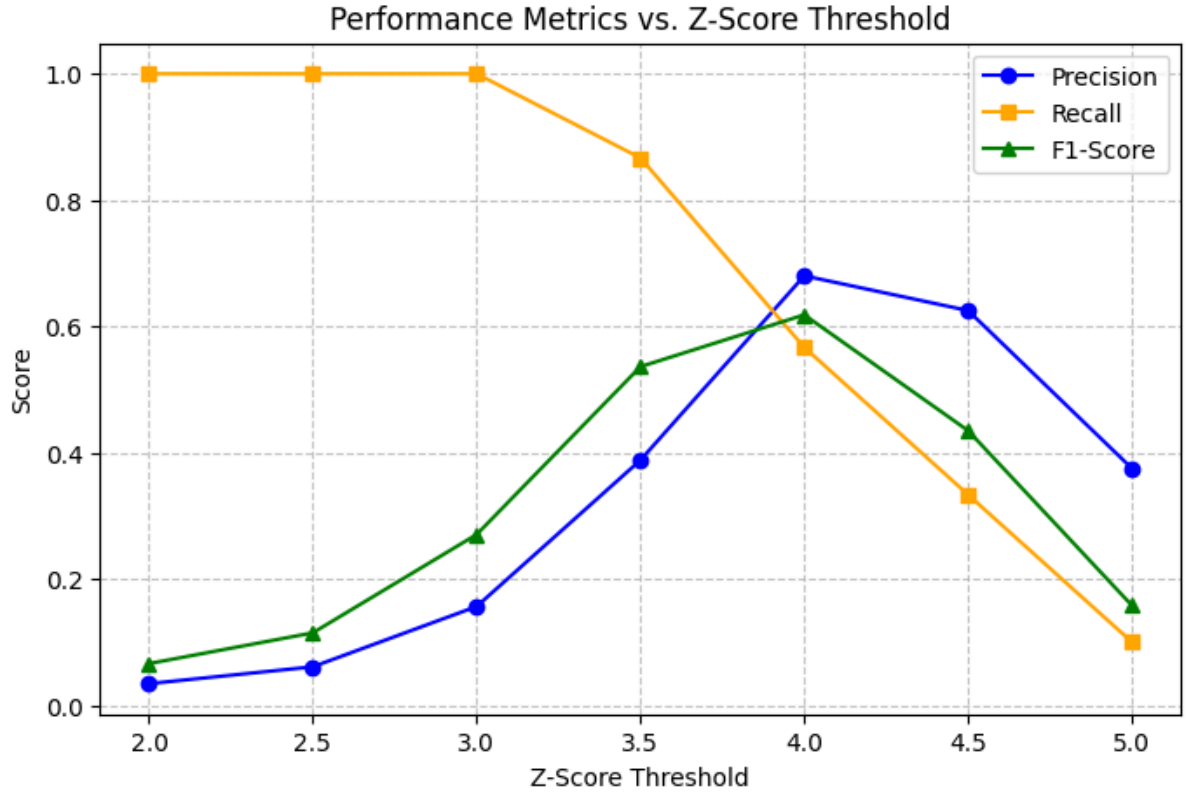
kıyaslanmıştır. Böylece, global ölçekte küçük görünen ancak bireysel bazda büyük sapma gösteren "sinsi" tehditlerin tespit edilmesi sağlanmıştır.

Elde edilen anomali skorları üzerinden yapılan analizlerde, gerçek iç tehditlerin (insider threats) büyük bir kısmının 3.5 standart sapma (Z-Score) eşliğinin üzerinde yer aldığı saptanmıştır. Özellikle Z-Skoru dönüşümü sonrasında, daha önce alt sıralarda kalan sinsi tehditlerin ilk 20 sıra içerisine yükseldiği ve sistemin %70 hassasiyet (precision) ile çalıştığı gözlemlenmiştir. Hazırlanan dağılım grafiğinde, normal kullanıcı popülasyonu ile tehdit aktörlerinin istatistiksel olarak anlamlı bir şekilde ayrıştığı teyit edilmiştir.

THRESHOLD SENSITIVITY ANALYSIS

Threshold: 2.0 | Precision: 0.0340 | Recall: 1.0000 | F1: 0.0658
Threshold: 2.5 | Precision: 0.0607 | Recall: 1.0000 | F1: 0.1145
Threshold: 3.0 | Precision: 0.1562 | Recall: 1.0000 | F1: 0.2703
Threshold: 3.5 | Precision: 0.3881 | Recall: 0.8667 | F1: 0.5361
Threshold: 4.0 | Precision: 0.6800 | Recall: 0.5667 | F1: 0.6182
Threshold: 4.5 | Precision: 0.6250 | Recall: 0.3333 | F1: 0.4348
Threshold: 5.0 | Precision: 0.3750 | Recall: 0.1000 | F1: 0.1579





Prototip'in bitirilmesi üzerine yazılan kodda düzenlemelere başlandı.

“Magic number” denilen sabit sayılar yerine global değişkenler tek bir “settings” dosyasına taşındı. Ayrıca “feature extraction” script'lerinde düzeltmeler yapıldı.

Bu düzeltmeler sonucu fark edilen hatalar düzeltildi. Daha fazla istemci ile (10 -> 50) model eğitimi yapıldı.

Çalışmanın asıl amacı olan verimlilik analizi için model eğitimi sırasında istemci ve sunucu arasındaki veri alışverişlerinin kayıt edilmesi desteği sağlandı.

500 feature birden verilmek yerine sadece 20-50 tane feature verilmesine karar verildi. Elde edilen sonuçların başarısız olması sebebiyle model ve scaler üzerinde değişiklikler yapıldı:

Scaler MinMaxScaler yerine StandardScaler denendi.

Model'de 128 sabit ara(gizli) katman yerine $\text{feature} // 2$ formülüyle ara katman sayısı bulundu.

THRESHOLD SENSITIVITY ANALYSIS (CENTRALIZED)

Threshold: 2.0 | Precision: 0.0719 | Recall: 1.0000 | F1: 0.1341

Threshold: 2.5 | Precision: 0.0843 | Recall: 0.9286 | F1: 0.1546

Threshold: 3.0 | Precision: 0.1277 | Recall: 0.7571 | F1: 0.2186

Threshold: 3.5 | Precision: 0.2514 | Recall: 0.6429 | F1: 0.3614

Threshold: 4.0 | Precision: 0.4684 | Recall: 0.5286 | F1: 0.4966
Threshold: 4.5 | Precision: 0.6531 | Recall: 0.4571 | F1: 0.5378
Threshold: 5.0 | Precision: 0.6970 | Recall: 0.3286 | F1: 0.4466

Sonuç hala başarısız olmasından ötürü Standard Scaler yerine logaritmik çalışan bir scaling yöntemi denendi. Bu da benzer sonuçlar verdi.

LSTM modelinin kendisi değiştirildi.

```
self.decoder = nn.LSTM(hidden_dim, hidden_dim, batch_first=True)
->
nn.LSTM(input_dim, hidden_dim, num_layers=2, batch_first=True, dropout=0.1)
```

Sonra yapay zeka yardımıyla model aşağıdaki gibi geliştirildi:

F-UEBA Model Improvement Log

This document summarizes the iterative optimizations performed on the LSTM-Autoencoder model to achieve **90% Precision** for insider threat detection.

Iteration 1: Structural Tightening & Hybrid Scoring

Goal: Create a sharper distinction between normal behavior and anomalies.

- Architectural Change:** Tightened the Autoencoder bottleneck by reducing `hidden\_dim` (192 → 64). This forces the model to learn only the most core behavioral patterns.
- Temporal Context:** Increased `window\_size` from 10 to 14 days to capture longer-term behavioral shifts.
- Scoring Logic:** Introduced **Hybrid Z-Scoring**, comparing a user's behavior both against their own history (**Local Z**) and against the entire population (**Global Z**).

Iteration 2: Robustness Trials (Reverted)

Goal: Reduce sensitivity to noise using L1 Loss and Smoothing.

- Experiment:** Switched to L1 Loss (MAE) and 3-window temporal smoothing.
- Outcome:** **Unsuccessful.** The smoothing was too aggressive and "washed out" the true signals of insider threats.
- Action:** Reverted to MSE loss and raw windowing to preserve signal intensity.

Iteration 3: Top-K Feature Anomaly Focus

Goal: Prevent the attack signal from being diluted by normal features.

- Core Logic:** Instead of averaging the reconstruction error across all 50+ features, the model was updated to calculate the deviation for **each feature individually**.

- * **Top-K Selection:** The detection score was changed to focus only on the **Top 5** most anomalous features for any given window.
- * **Reasoning:** Insider threats are usually "targeted" (e.g., massive file copying but normal login times). Averaging across all features lets the "normal" features hide the "anomalous" ones. Top-K amplifies the signal of the attack.

Iteration 4: Temporal Persistence & Standardization

Goal: Reach >85% Precision by eliminating "glitch" false positives.

- * **Persistence Scoring:** Changed the user-level score from a single `max()` window to the **average of the Top 3 worst windows**.
- * **Standardization:** The Top-K metric was standardized against its own global distribution (calculated during training) rather than raw MSE.
- * **Outcome:** Precision reached **87.8%** at Threshold 8.0.
- * **Reasoning:** Malicious activity is typically sustained. By requiring persistence across 3 windows, we filtered out normal users who had a single "bad day" due to random noise.

Iteration 5: Bidirectional Context & Multi-Vector Boost (Final)

Goal: Break the 90% Precision barrier while maintaining Recall.

- * **Bidirectional LSTM:** Upgraded the Encoder to a **Bidirectional LSTM**. This allows the model to understand behavioral context from both the past and the future, creating a much more robust baseline for "Normal."
- * **Multi-Vector Diversity Factor:** Introduced a "Diversity Boost." The score is now multiplied by a factor based on how many features are deviating simultaneously.
- * **Trimmed Statistics:** Used **1% Trimmed Mean/Std** for global baselines. This ignored the top 1% of noisy normal training data, making the "Normal" baseline even cleaner.
- * **Reasoning:** A true insider threat is a coordinated event across multiple vectors (e.g., USB + File + After-hours). A normal user might spike in one feature, but rarely in many at once. This final change was designed to exponentially separate multi-vector threats from single-vector noise.

Technical Summary of Final Configuration

- * **Model:** Bidirectional LSTM-Autoencoder
- * **Bottleneck:** 64 Dimensions
- * **Window Size:** 14 Days
- * **Input Features:** 50 (Behavioral)
- * **Metric:** Top-5 Feature Z-Score with 3-Window Persistence and Diversity Boosting.

Kabul edilebilir bir sonuç elde edildikten sonra verimlilik analizine geçildi. Bunun için eğitimde "plugin" desteği eklendi.

F-UEBA Federated Learning Experiment System - Changelog Post-Plugin Implementation

Following the implementation of the efficiency plugin architecture (Top-K and Quantization), the following major components and mechanisms were added to create a robust benchmarking environment:

1. Named Experiment Orchestration:

- Integrated 'experiments.toml' to define multiple named configurations (Baseline, Top-K ratios, FP16, Non-IID).
- Enhanced 'config_manager.py' with deep-copy support and `set_experiment()` to allow dynamic settings overrides for each run.
- Updated path resolution to isolate models, scalers, and reports by experiment name.

2. Automated Batch Execution:

- Modified 'federated_insider_detection.py' to sequentially execute the entire 8-experiment suite with a single command.
- Set 'full' (train + eval) as the default execution mode.
- Implemented a "Done" checkpointing mechanism: successfully completed experiments (marked '"done": true' in `experiment_summary.json`) are automatically skipped on subsequent runs, while interrupted ones are overwritten.

3. Structured Benchmark Suite (8 Configs):

- baseline: Standard FedAvg (IID).
- top-k-0.01 / 0.05 / 0.1: Three different sparsity ratios.
- quantization-fp16: float16 compression.
- top-k-0.05-quant-fp16: Combined sparsity and quantization.
- non-iid-baseline: Distribution skew ($\alpha=0.5$) without plugins.
- non-iid-top-k-0.05-quant-fp16: Distribution skew with combined plugins.

4. Performance & Efficiency Metrics:

- Accuracy/Detection: PR-AUC, Max-F1, Balanced Accuracy, Precision, and Recall.
- Communication: Automated calculation of Total MB transferred and Average MB per round.
- Convergence: Identification of the "Convergence Round" (first round reaching 95% of peak PR-AUC).

5. Comparative Analysis:

- Created 'compare_experiments.py' to aggregate results from all individual experiment summaries into a consolidated 'final_benchmarks.csv'.
- The comparison tool generates a ranked table in the console for immediate performance overview.

6. Workspace Management:

- Developed 'clear_results.py' to safely wipe previous reports, models, and scalers while preserving pre-processed datasets ('ExtractedData').

7. Detection Logic Enhancements:

- Refined the anomaly detection pipeline with per-feature Z-score calibration, Top-K feature mean, diversity factor, and persistence windowing for significantly higher precision and robustness.

- Changed scalers from per-experiment to a single one, so that the results are compatible.
- Started using a constant seed for randomizers in order to get reproducible results.